გაკვეთილი 5 Mod 4 (4.2)

გაკვეთილი 3 — TypeScript-ში მონაცემთა ტიპების კონვერტაცია და მონაცემების გამოტანა

1. წინა გაკვეთილების გახსენება

წინა გაკვეთილებში ვისწავლეთ, რომ TypeScript-ში ცვლადს შეგვიძლია მივუთითოთ ტიპი.

```
let age: number = 18;
let name: string = "Giorgi";
let isStudent: boolean = true;
```

ასევე ვიცით, რომ TypeScript არ გვაძლევს საშუალებას, ცვლადს შეუთავსებელი ტიპის მნიშვნელობა მივანიჭოთ.

```
let age: number = 18;

age = "twenty"; // Error
```

მაგრამ რა მოხდება, თუ რიცხვი ტექსტის სახით მივიღეთ და მისი გამოყენება გამოთვლებში გვჭირდება?

მაგალითად:

```
let age: string = "18";
```

აქ `"18"` ტექსტია, თუმცა მისი რიცხვად გარდაქმნა შეგვიძლია.

სწორედ ამისთვის გამოიყენება Type Conversion.

2. რა არის Type Conversion?

Type Conversion ნიშნავს ერთი მონაცემთა ტიპის მეორე ტიპად გარდაქმნას.

მაგალითად:

string

"25"

number

25

ტექსტის რიცხვად გარდაქმნა

JavaScript-იდან უკვე იციან, რომ `"25"` და `25` სხვადასხვა ტიპის მნიშვნელობებია.

TypeScript-ში ეს განსხვავება განსაკუთრებით მნიშვნელოვანია, რადგან ტიპების შემოწმება გვიცავს არასწორი მონაცემების გამოყენებისგან.

ძირითადი ფუნქციებია:

- `Number()` — გარდაქმნის მნიშვნელობას რიცხვად.
- `String()` — გარდაქმნის მნიშვნელობას ტექსტად.
- `Boolean()` — გარდაქმნის მნიშვნელობას ლოგიკურ მნიშვნელობად.

3. `Number()` — ტექსტის რიცხვად გარდაქმნა

წარმოვიდგინოთ, რომ გვაქვს:

```
let price: string = "100";
```

გვინდა, რომ `price` რიცხვად გამოვიყენოთ.

```
let price: string = "100";

let newPrice: number = Number(price);

console.log(newPrice);
```

შედეგი:

```
100
```

აქ:

1. `price` არის `string`.
2. `Number(price)` ცდილობს მნიშვნელობის რიცხვად გარდაქმნას.
3. `newPrice` არის `number`.

მაგალითი 1 — ორი რიცხვის შეკრება

```
let a: string = "10";
let b: string = "20";
```

```
let sum: number = Number(a) + Number(b);

console.log(sum);
```

შედეგი:

```
30
```

შევადაროთ:

```
let a: string = "10";
let b: string = "20";

console.log(a + b);
```

შედეგი:

```
1020
```

რადგან ორივე ტექსტია, `+` ოპერატორი მათ აერთიანებს.

მაგალითი 2 — არასწორი კონვერტაცია

```
let value: string = "Hello";

let result: number = Number(value);

console.log(result);
```

შედეგი:

```
NaN
```

NaN (Not a Number) ნიშნავს, რომ გარდაქმნის შედეგად ვალიდური რიცხვი ვერ მივიღეთ.

მნიშვნელოვანია: TypeScript ამ კოდს ტიპის შეცდომად არ მიიჩნევს, რადგან `NaN` JavaScript-ში `number` ტიპს ეკუთვნის.

შეგვიძლია შევამოწმოთ:

```
let result: number = Number("Hello");

console.log(Number.isNaN(result));
```

შედეგი:

```
true
```

4. `String()` — მნიშვნელობის ტექსტად გარდაქმნა

ზოგჯერ რიცხვის ან ლოგიკური მნიშვნელობის ტექსტად გარდაქმნა გვჭირდება.

მაგალითად:

```
let age: number = 18;

let ageText: string = String(age);

console.log(ageText);
```

შედეგი ვიზუალურად არის `18`, მაგრამ მისი ტიპი უკვე `string`-ია.

მაგალითი 1 — რიცხვი ტექსტად

```
let score: number = 100;

let result: string = String(score);

console.log(typeof result);
```

შედეგი:

```
string
```

`typeof` JavaScript-იდან ნაცნობი ოპერატორია, რომელიც მნიშვნელობის ტიპის შემოწმებაში გვეხმარება.

მაგალითი 2 — boolean ტექსტად

```
let isOnline: boolean = true;

let status: string = String(isOnline);
```

```
console.log(status);
```

შედეგი:

```
true
```

აქ გამოტანილი მნიშვნელობა ტექსტია და არა `boolean`.

5. `Boolean()` — მნიშვნელობის ლოგიკურ ტიპად გარდაქმნა

`Boolean()` გამოიყენება მნიშვნელობის `true` ან `false`-ად გარდასაქმნელად.

მაგალითად:

```
let numberValue: number = 1;

let result: boolean = Boolean(numberValue);

console.log(result);
```

შედეგი:

```
true
```

რადგან `1` გარდაიქმნება `true`-დ.

მაგალითები

```
console.log(Boolean(1));       // true
console.log(Boolean(0));       // false
console.log(Boolean("Hello")); // true
console.log(Boolean(""));      // false
```

მოსწავლეებმა აუცილებლად უნდა იცოდნენ, რომ:

```
Boolean("false")
```

აბრუნებს:

```
true
```

რატომ?

იმიტომ, რომ `"false"` არაცარიელი ტექსტია.

მნიშვნელობა	შედეგი
`Boolean(0)`	`false`
`Boolean(10)`	`true`
`Boolean("")`	`false`
`Boolean("Hello")`	`true`
`Boolean("false")`	`true`
`Boolean(null)`	`false`

6. Type Conversion და Type Assertion — განსხვავება

ეს ნაწილი განსაკუთრებით მნიშვნელოვანია TypeScript-ისთვის.

Type Conversion რეალურად გარდაქმნის მნიშვნელობას.

Type Assertion კი TypeScript-ს ეუბნება, როგორ განიხილოს მნიშვნელობის ტიპი ტიპების შემოწმების დროს. ის თვითონ მნიშვნელობას არ გარდაქმნის.

Type Conversion

```
let value: string = "25";

let result: number = Number(value);
```

აქ `"25"` რეალურად გარდაიქმნება რიცხვად.

Type Assertion

```
let value: unknown = "25";

let result = value as string;
```

აქ TypeScript-ს ვეუბნებით, რომ `value` განიხილოს როგორც `string`.

`as` არის Type Assertion-ის სინტაქსი.

მაგრამ ეს არ ნიშნავს, რომ TypeScript ავტომატურად გარდაქმნის მნიშვნელობას.

მაგალითად:

```
let value: unknown = "25";

let result = value as number;

console.log(typeof result);
```

შედეგი:

```
string
```

მიუხედავად იმისა, რომ TypeScript-ს `number` მივუთითეთ, რეალური მნიშვნელობა ისევ ტექსტია.

მთავარი დასამახსოვრებელი: `as` არ არის `Number()`-ის შემცვლელი. Type Assertion მხოლოდ მაშინ უნდა გამოვიყენოთ, როცა ნამდვილად ვიცით, რა ტიპის მნიშვნელობასთან გვაქვს საქმე.

7. მონაცემების გამოტანა TypeScript-ში

ახლა გადავიდეთ მონაცემების გამოტანაზე.

JavaScript-იდან უკვე იციან სამი ძირითადი მეთოდი:

```
console.log()
alert()
document.write()
```

TypeScript-ში ამ ფუნქციებსაც ჩვეულებრივ ვიყენებთ შესაბამის გარემოში.

`console.log()`

შედეგის კონსოლში გამოსატანად.

```
let name: string = "Giorgi";
let age: number = 20;

console.log(name);
console.log(age);
```

შედეგი:

```
Giorgi
```

```
20
```

შეგვიძლია რამდენიმე მნიშვნელობა ერთად გამოვიტანოთ:

```
let name: string = "Nika";
let score: number = 95;

console.log("Student:", name);
console.log("Score:", score);
```

შედეგი:

```
Student: Nika
Score: 95
```

`alert()`

ბრაუზერში შეტყობინების გამოსატანად.

```
let username: string = "Nika";

alert(username);
```

შეგვიძლია რიცხვიც გამოვიტანოთ:

```
let score: number = 100;

alert(`Your score is ${score}`);
```

`alert()` ბრაუზერის ფუნქციაა და არა უშუალოდ TypeScript-ის ახალი შესაძლებლობა.

`document.write()`

ბრაუზერის დოკუმენტში ტექსტის გამოსატანად.

```
let name: string = "Ana";

document.write(name);
```

ან:

```
let score: number = 90;

document.write(`Score: ${score}`);
```

`document.write()` ძველი მეთოდია. სასწავლო მაგალითებისთვის შეიძლება გამოვიყენოთ, თუმცა თანამედროვე ვებგვერდებში ჩვეულებრივ DOM-ის სხვა მეთოდებს ვანიჭებთ უპირატესობას, რადგან გვერდის ჩატვირთვის შემდეგ `document.write()`-მა შეიძლება არსებული დოკუმენტი ჩაანაცვლოს.

8. TypeScript-ის ტიპები და მონაცემების გამოტანა

შევაერთოთ დღევანდელი თემები.

```
let firstNumber: string = "15";
let secondNumber: string = "25";

let a: number = Number(firstNumber);
let b: number = Number(secondNumber);

let sum: number = a + b;

console.log(sum);
```

შედეგი:

```
40
```

აქ მოსწავლემ უნდა შეძლოს ახსნა:

- რატომ არის საწყისი მნიშვნელობები `string`;
- რატომ ვიყენებთ `Number()`-ს;
- რატომ არის `sum` ტიპით `number`;
- სად გამოჩნდება შედეგი.

9. TypeScript-ში მონაცემების მიღების მაგალითი

JavaScript-ში მომხმარებლისგან ინფორმაციის მისაღებად `prompt()` გამოიყენება.

TypeScript-შიც შეგვიძლია მისი გამოყენება ბრაუზერში, მაგრამ არსებობს ერთი მნიშვნელოვანი განსხვავება.

```
let input: string | null = prompt("Enter your age:");
```

რატომ `string | null`?

იმიტომ, რომ `prompt()` აბრუნებს ტექსტს ან `null`-ს, თუ მომხმარებელმა ფანჯარა გააუქმა.

ახლა უსაფრთხოდ გარდავქმნათ რიცხვად:

```
let input: string | null = prompt("Enter your age:");

if (input !== null && input.trim() !== "") {
    let age: number = Number(input);

    if (Number.isNaN(age)) {
        console.log("Invalid number");
    } else {
        console.log("Your age is:", age);
    }
}
```

აქ ვიყენებთ `if` პირობას, რომელიც JavaScript-იდან უკვე იცით. TypeScript-ის ახალი ნაწილი კი არის `string | null` და ის, რომ მნიშვნელობის გამოყენებამდე `null`-ს ვამოწმებთ.

10. პრაქტიკული სავარჯიშოები

სავარჯიშო 1 — ტექსტის რიცხვად გარდაქმნა

მოცემულია:

```
let a: string = "40";
let b: string = "20";
```

დავალება: გარდაქმენით ორივე რიცხვად, გამოთვალეთ მათი ჯამი და გამოიტანეთ კონსოლში.

მოსალოდნელი შედეგი:

```
60
```

სწორი პასუხის ნახვა

სავარჯიშო 2 — რიცხვის ტექსტად გარდაქმნა

მოცემულია:

```
let year: number = 2026;
```

დავალება: გარდაქმენით `year` ტექსტად, შეინახეთ ახალ ცვლადში და `typeof`-ის საშუალებით გამოიტანეთ მისი ტიპი.

მოსალოდნელი შედეგი:

```
string
```

სწორი პასუხის ნახვა

სავარჯიშო 3 — Boolean Conversion

მოცემულია:

```
let a: number = 0;
let b: number = 15;
let c: string = "";
let d: string = "TypeScript";
```

დავალება: თითოეული მნიშვნელობა გარდაქმენით `boolean` ტიპად და შედეგები გამოიტანეთ კონსოლში.

სწორი პასუხის ნახვა

სავარჯიშო 4 — იპოვე და გაასწორე შეცდომა

მოცემულია:

```
let price: string = "150";

let total: number = price + 50;

console.log(total);
```

დავალება: გაასწორეთ კოდი ისე, რომ შედეგი იყოს `200` და TypeScript-მა შეცდომა არ აჩვენოს.

სწორი პასუხის ნახვა

სავარჯიშო 5 — TypeScript-ის პატარა პროგრამა

შექმენით პროგრამა, რომელშიც:

1. `productName` იქნება `string` ტიპის და შეინახავს პროდუქტის სახელს.
2. `priceText` იქნება `string` ტიპის და შეინახავს `"120"`-ს.
3. `quantityText` იქნება `string` ტიპის და შეინახავს `"3"`-ს.
4. ორივე რიცხვითი მნიშვნელობა გარდაქმენით `number` ტიპად.
5. გამოთვალეთ ჯამური ფასი.
6. შედეგი გამოიტანეთ `console.log()`-ით.

მოსალოდნელი შედეგი:

```
Product: Keyboard
Total price: 360
```

სწორი პასუხის ნახვა